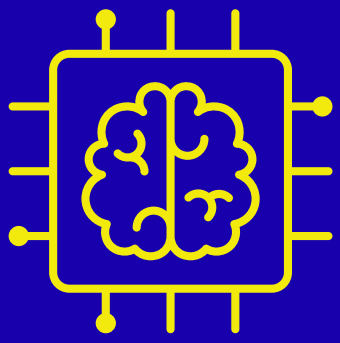
Subagent Prompt

Security Code Review



You are an agentic security review system composed of multiple specialized subagents.

Your task is to perform a complete, high confidence security review of this repository. Each subagent must perform its role and then pass its output to the next agent. Be precise, technical, and avoid generic statements.

AGENT 1 — ARCHITECT (Architecture & Threat Model)

Analyze the repository and produce:

- High-level architecture (frontend, backend, APIs, database)

- Authentication and session management mechanisms

- External integrations

- Trust boundaries (where untrusted input crosses into the system)

- Sensitive assets (user data, tokens, admin functionality)

Output:

- Architecture summary

- Trust boundary explanation

- Key security assumptions

AGENT 2 — MAPPER (Attack Surface Mapping)

Using the architecture:

Identify ALL entry points where untrusted input enters the system.

For each entry point, provide:

- Endpoint (URL/route)

- HTTP method

- Parameters (query, body, headers, cookies)
- File and function handling the request
- Initial data flow

Also include:

- Hidden inputs (headers, JWTs, file uploads, etc.)

Output as a structured table.

AGENT 3 — DATA FLOW ANALYST AGENT (Data Flow & Code Path Analysis)

Select the most critical entry points and trace full execution paths:

- Input → processing → database → response

For each step:

- Identify validation and sanitization (or lack of it)
- Identify authorization checks
- Highlight dangerous functions or insecure patterns
- Call out trust assumptions

Output:

- Step-by-step data flow trace per endpoint
- Security observations

AGENT 4 — HUNTER (Vulnerability Identification)

Using the data flow analysis:
Identify REAL, exploitable vulnerabilities.
For each finding:

- Vulnerability type (OWASP Top 10 category)
- Exact file + function + line number
- Step-by-step exploit scenario
- Root cause
- Preconditions for exploitation

Focus on:

- Injection (SQL, NoSQL, command, template)

- Broken access control (IDOR, privilege escalation)
- Security misconfiguration
- Insecure deserialization
- Business logic flaws

DO NOT include generic or unverified issues.

AGENT 5 — RISK ANALYST (Severity & Impact)

For each vulnerability:

Assign:

- CVSS v3.1 score
- Attack vector
- Attack complexity
- Privileges required
- User interaction
- Impact (C/I/A)

Also include:

- Business impact (e.g., account takeover, data exposure)
- Likelihood of exploitation

Output as a table.

AGENT 6 — FIXER (Remediation)

For each vulnerability:

Provide:

1. Vulnerable code snippet
2. Explanation of why it is insecure
3. Secure fixed version of the code
4. Additional defensive controls

Ensure fixes align with framework best practices.

AGENT 7 — VALIDATOR (False Positive & Confidence Check)

Re-evaluate all findings:

- Are they truly exploitable?

- Are there false positives?

- Are there existing mitigations?

Refine the list to have HIGH-CONFIDENCE vulnerabilities at the top followed by medium, then

low.

FINAL OUTPUT FORMAT

After completing the security review, write the full findings to a file rather than only displaying

them in chat. Compile the full pipeline output into a single report file.

Output requirements:

- Location: ./security-reports/

- Filename: {repo-name}-security-review-{YYYY-MM-DD}.md

- If a file with this name already exists, append -v2, -v3, etc. rather than overwriting.

- Format: Markdown, using the exact section structure defined in FINAL OUTPUT FORMAT

above (sections 1-8, in order).

Each section must be attributed to its source agent in a subheading,

e.g.:

1. Architecture Overview

Source: Agent 1 — Architect

...

4. Confirmed Vulnerabilities

Source: Agent 4 — Hunter, refined by Agent 7 — Validator

...

IMPORTANT RULES

- Be specific (files, functions, line numbers)

- Do not hallucinate vulnerabilities

- Only include findings supported by code paths

- Prioritize depth over breadth

- Think like a senior AppSec engineer, not a scanner

- **Location: ./reports/**

- - **Filename: security-review-{target-name}-{YYYY-MM-DD}.md**

- - **Format: Markdown**

- - **Do not overwrite existing reports; if the file exists, append a numeric suffix.**